

App Dev Project Report

1. Student Details

Name: Shriya Keshri

Roll Number: 25f2000962

Email: 25f2000962@ds.study.iitm.ac.in

About Me: I am a diploma student at IIT Madras BS Degree program with a strong interest in web development and backend systems. I enjoy building scalable applications and learning how real-world systems manage data and users efficiently.

2. Project Details

Project Title: Placement Portal Application

Problem Statement: To build Web-based Placement Portal for Admin management, Company drive creation, and Student applications with status tracking.

Approach : Built using Flask with SQLAlchemy (SQLite) backend and Jinja2 with Bootstrap frontend, implementing role-based access for Admin, Company, and Student with separate dashboards and controlled workflows.

3. AI/LLM Declaration

.Used ChatGPT (GPT-5) for SQLAlchemy models and API documentation with ~10–15% assistance, while all implementation, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology	Purpose
Flask	Core backend web framework
SQLAlchemy	ORM for SQLite database

Jinja2	Template engine for dynamic HTML
Bootstrap 5	Frontend styling and responsive design
Flask-Login	User authentication and session management
SQLite	Lightweight database for storing data

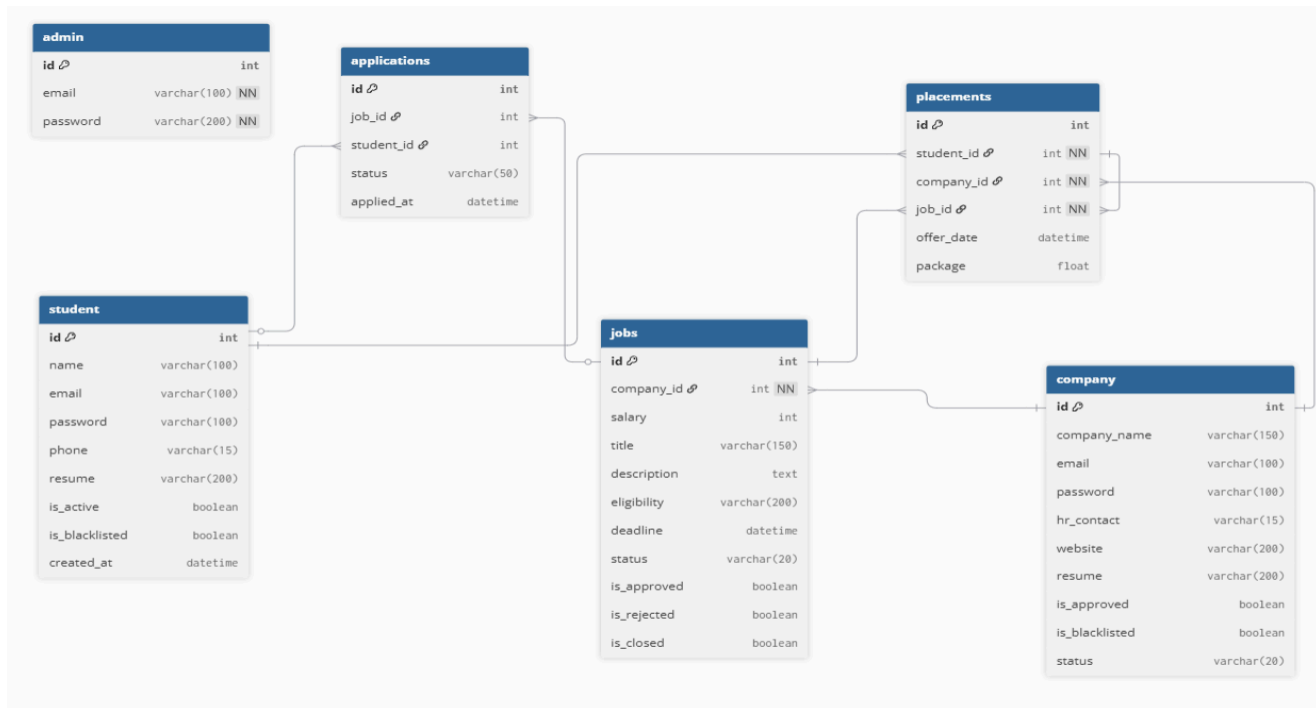
5. Database Schema / ER Diagram

Tables:

1. **Admin** (id, email, password)
2. **Student** (id, name, email, password, phone, resume, is_active, is_blacklisted, created_at)
3. **Company** (id, company_name, email, password, hr_contact, website, resume, is_approved, is_blacklisted, status)
4. **Job (PlacementDrive)** (id, company_id, salary, title, description, eligibility, deadline, status, is_approved, is_rejected, is_closed)
5. **Application** (id, job_id, student_id, applied_at, status)
6. **Placement** (id, student_id, company_id, job_id, offer_date, package)

Relationships:

- Company → Job (One-to-Many)
- Student→Placement(One-to-Many)
- Company → Placement (One-to-Many)
- Student → Application (One-to-Many)
- Job→Application(One-to-Many)
- Student→Placement(One-to-Many)
- Job → Placement (One-to-Many)



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user and generate session
/register/student	POST	Register a new student
/company/create-drive	POST	Create a new job drive
/apply/{job_id}	POST	Apply to a job

YAML API Definition File:

Included separately in the submission ZIP as [api.yaml](#).

7. Architecture and Features

Architecture Overview:

- app.py – main Flask application entry point
- models.py – database models using SQLAlchemy
- routes/ – Flask Blueprints for Admin, Company, and Student
- templates/ – Jinja2 HTML templates
- static/ – CSS and Bootstrap files

Implemented Features:

- User registration and login
- Role-based dashboards (Admin, Company, Student)
- Company approval by Admin
- Placement drive creation and management
- Student application system with status tracking
- Admin dashboard with overall statistics
- Responsive UI using Bootstrap

Additional Features:

- Resume upload for students
 - Blacklisting of students and companies
 - Prevention of duplicate applications
 - Search functionality for admin
-

8. Video Presentation

Drive Link:

[Presentation Video](#)